

Technical Report For Granite Guardian

Ines Mouine - smjg43

Krishna Chauhan - mknf66

Rayane El-Mselmi - xvns66

Adam Fallon - vtwn33

Ewan Wong - fvcg54

Adel Lawgaly - jlsg53

Section 1: Introduction

1.1 Project summary: what is Granite Guardian and what does it do?

For an average driver, the dashboard of a car can be a source of stress. When a "Check Engine" light flickers on, it rarely provides context, leaving the user caught between the fear of a breakdown and the confusion of technical jargon. **Granite Guardian** was born from the desire to bridge this gap, turning cold, complex vehicle data into a supportive and transparent conversation.

Our system allows users to upload **OBD-II** CSV files. Once uploaded, the platform performs an analysis of the vehicle’s vital signs (ex: engine RPM, catalytic converter temperatures, or coolant temperature). The client requested CSV files, though this system could easily be extended to work with live data.

Instead of just flagging a numerical anomaly, our integration of the **IBM Granite 3.0 2B** model allows the system to speak to the user, explaining why a value is concerning and what steps should be taken next. The goal is to move from reactive repairs to proactive care and saving users money and providing peace of mind

1.2: Access and Technical Setup

The system was developed to run entirely on the user's local machine or on an embedded system. This ensures that sensitive vehicle diagnostic logs (which could technically reveal driving patterns or locations) never leave the user's hardware, and it mitigates the need for a constant Wi-Fi connection.

- The frontend was developed using React 19 and **TypeScript** which provides a "chic," high-contrast dashboard designed for readability in various lighting conditions. It utilizes Lucide-React for intuitive iconography.
- The backend is powered by a FastAPI server organised into a modular architecture with dedicated route, service, model, and middleware layers. It uses SQLAlchemy as an ORM to communicate with a local SQLite database. The middleware stack includes centralised error handling, security headers, rate limiting (200 requests/minute per IP via slowapi), and request logging.

About the required Environment for Launch:

To ensure the system operates correctly, you must activate the following environment:

- **Python 3.12+** Virtual Environment containing dependencies like fastapi, uvicorn and pandas.
- The **Ollama** application must be running in the system tray to handle AI requests.
- And **Node.js** Environment: To serve the React frontend via Vite

1.3: Status of Behavioural Requirements

ID	Requirement	State	Justification /Commentary

BR1	CSV File Ingestion	Completed	allows for uploading CSV files.
BR2	Anomaly detection	Completed	detects anomalies in engine coolant sensor (e.g. noise).
BR3	Granite AI Chatbot	Completed	Local integration via Ollama for natural-language explanations. Supports per-alert and full-summary modes with a structured fallback when Granite is unavailable.
BR4	Alerts & Filtering	Completed	Alert retrieval with filtering by severity, sensor type, and acknowledgement status. Users can acknowledge and un-acknowledge individual alerts

Table 1: Status of Behavioral Requirements

Table 1 provides an overview of the system’s core functionalities as defined in the Requirements Specification. It tracks the successful implementation of each feature, ensuring alignment between the initial user stories and the final software deliverable

Section 2: Technical Development

2.1 Technologies, Platforms and System Architecture

To fulfill the requirements (delivering a responsive and secure application), Granite Guardian was built using a modern three-tier architecture. This separation ensures that the user interface, business logic and data storage can operate and scale independently:

1. The frontend

The user interface is developed as a single page application in which we included:

- React for its efficient, component-based architecture.
- Typescript to enforce type checking and thus reduce runtime errors during development.
- The usage of React for its component-based architecture which allowed us to build reusable UI elements (such as the Chatbot interface and the data dashboards) and Typescript to enforce type-checking to reduce runtime errors during development.

2. The backend

The backend is implemented as a FastAPI web service. It was restructured from an initial flat-file layout (a single main.py with all routes and a helpers.py with mixed utilities) into a modular architecture with clearly separated concerns:

- **Route layer (routes/):** Five route modules (upload_routes, data_routes, diagnostics_routes, alert_routes, granite_routes) handle HTTP concerns such as parameter parsing and status codes. All business logic is delegated to the service layer.
- **Service layer (services/):** Five service modules contain the core business logic. upload_service handles file validation and CSV cleaning. data_service provides sensor data retrieval with pagination and summary statistics. diagnostics_service runs multi-sensor anomaly detection against configurable thresholds. alert_service manages alert filtering, acknowledgement, and log clearing. granite_service handles Granite natural-language generation with a fallback engine. A shared validators module centralises filename safety checks, path traversal prevention, and sensor name validation.
- **Model layer (models/):** Pydantic schemas (schemas.py) provide automatic request body validation. Warning classes (warning.py) define typed warnings for each sensor with a Severity enum (low, medium, high, critical) and contextual human-readable messages.
- **Middleware layer (middleware/):** Four middleware components registered in main.py: centralised error handling ensuring consistent JSON error responses; security headers (X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, HSTS, Referrer-Policy, Permissions-Policy, Cache-Control); rate limiting via slowapi at 200 requests/minute per IP; and request logging recording method, path, status code, and duration for every request.
- **Configuration (config.py):** Centralises all constants including 12 recognised OBD-II sensor names, anomaly thresholds with min/max ranges for 7 sensors, Granite model settings, and pagination defaults.

Additional backend technologies include NumPy and pandas for CSV data handling, scikit-learn for ML classifiers, and SQLAlchemy as an ORM to interact with a local SQLite database.

3. The Diagnostic & AI Inference Layer

The AI engine is hosted entirely on the user's hardware (rather than relying on external APIs):

- Ollama Framework: used to serve as the local execution environment for our LLM (large language model).
- IBM Granite 3.0 2B (granite3-dense:2b): We chose to base the Chatbot on this model so when the backend detects a mechanical anomaly, it constructs a prompt containing the error context and sends it to the local Granite model. The model then generates a natural-language explanation tailored for a non-expert driver, which is routed back to the React frontend.

4. The Data Persistence Layer

To store the users sessions, his historic of its diagnostic alerts and system configurations we implemented the following solutions:

- **SQLite:** Chosen because our system emphasises local deployment ("Privacy by Design"),

SQLite allows the entire database to be stored as a single local file (.db) without requiring the user to configure complex external database servers (ex PostgreSQL, MySQL..).

- And the SQLAlchemy: we used it as our Object/Relational Mapper. It abstracts the raw SQL queries into Python objects, preventing SQL injection attacks and ensuring the data access layer remains secure and maintainable.

2.2 Backend Architecture in Detail

The backend API exposes five endpoint groups, each backed by a dedicated service module.

Upload and Validation Pipeline The upload endpoint (/uploads/) enforces a strict validation sequence: filename extension check, content-type verification against multiple allowed MIME types, filename safety validation (path traversal prevention, length limits), duplicate detection, empty file rejection, and CSV content validation. The content validator parses the uploaded CSV, checks for at least one recognised OBD-II sensor column from a configurable list of twelve, removes duplicate rows, and sorts data by engine run time when present. Invalid files are cleaned up from disk automatically on failure.

Multi-Sensor Anomaly Detection The diagnostics endpoint (/diagnostics/{filename}) scans every row of a CSV against configurable min/max thresholds for six sensors: fuel tank level, engine coolant temperature, engine RPM, throttle position, engine load, and intake air temperature. Each violation generates a typed warning object with a severity level determined by the reading (for example, coolant temperature above 120°C is classified as critical, while above 110°C is high). Results are cached to log files and can be force-rescanned. A summary aggregating warnings by type and severity is included in every response.

Users can disable individual sensors from detection via /diagnostics/sensors/disable, addressing the requirement (BR2.3) that malfunctioning sensors should not trigger false positives.

Alert Management The alerts endpoint group (/alerts/) provides retrieval of warnings per file or across all files, with optional filtering by severity, sensor type, and acknowledgement status. Users can acknowledge individual alerts to track which issues have been reviewed, and un-acknowledge them if needed. Per-file and bulk log clearing are also supported. This directly implements the user stories for filtering and acknowledging alerts (BR7.1–BR7.3).

Natural-Language Explanations The explanation endpoint (/explain/{filename}) generates plain-language diagnostics via IBM Granite. The prompt builder formats warning context differently for single-alert explanations versus full-file summaries. When Granite is unavailable (Ollama not running or model not loaded), the system returns a structured fallback explanation that groups issues by severity and provides appropriate urgency messaging. The response always indicates whether the explanation came from Granite or the fallback engine.

Data Retrieval and Pagination The data endpoint (/data/{filename}) supports two access modes: evenly-spaced sampling (for chart-friendly data reduction) and paginated retrieval with configurable page size. A summary sub-endpoint (/data/{filename}/summary) returns descriptive statistics (min, max, mean, standard deviation, null count) for every numeric column, supporting the frontend's need to display sensor trends.

2.3 API Documentation

A comprehensive API documentation file (API_DOCS.md) was produced alongside the codebase. It includes a quickstart guide with installation and launch commands, the full project directory structure, detailed endpoint documentation with example curl commands for every route, expected response shapes, error codes, and a manual test checklist covering the complete upload-to-explanation workflow.

2.4 Security Considerations

Security was addressed at multiple layers of the backend to protect against common web vulnerabilities and API abuse.

The security headers middleware applies seven protective headers to every HTTP response. X-Content-Type-Options prevents browsers from MIME-type sniffing, which could allow an attacker to trick the browser into executing uploaded CSV files as scripts. X-Frame-Options blocks the application from being embedded in iframes, preventing clickjacking attacks. X-XSS-Protection enables the legacy cross-site scripting filter for older browsers. Strict-Transport-Security enforces HTTPS connections when deployed to production, preventing man-in-the-middle attacks. Referrer-Policy controls how much referrer information is leaked in outgoing requests. Permissions-Policy restricts access to browser features such as camera, microphone, and geolocation that the application does not need. Cache-Control is set to no-store to prevent caching of sensitive diagnostic data.

Rate limiting is enforced at 200 requests per minute per client IP address using slowapi. This threshold is generous enough for normal use but prevents automated abuse, denial-of-service attempts, or runaway frontend request loops. When a client exceeds the limit, they receive a 429 Too Many Requests response.

Filename validation is enforced on every endpoint that accepts a filename parameter. The shared validator rejects empty filenames, filenames containing path traversal sequences (.. / or), filenames that do not end with .csv, and filenames exceeding 255 characters. This prevents directory traversal attacks where a malicious filename like ../../etc/passwd could be used to access files outside the upload directory.

On the database side, SQLAlchemy's ORM approach prevents SQL injection by parameterising all queries rather than constructing raw SQL strings. User passwords are securely hashed before storage, and authentication uses JSON Web Tokens (JWT) for stateless session management.

2.5 Error Handling Strategy

A centralised error handling middleware ensures that every error response from the API follows a consistent JSON structure regardless of the error type. This was a deliberate design decision to make the API predictable for the frontend team, who could rely on a single response shape for all error cases.

HTTP exceptions (such as 404 Not Found or 409 Conflict) return a JSON body containing the detail

message and the status code. Pydantic validation errors, which occur when request bodies fail schema validation, are caught and transformed into a 400 response that includes an array of individual field errors, each specifying the field name, error message, and validation type. Unhandled exceptions are caught by a general handler that logs the full stack trace for debugging but returns only a generic "An unexpected error occurred" message to the client, avoiding the leakage of internal implementation details.

All errors are logged with the request method, URL path, and status code, providing an audit trail for debugging without exposing sensitive information in responses.

2.6 Design Patterns and Principles

Several software engineering principles guided the backend development.

Separation of concerns was enforced by dividing the codebase into distinct layers. Route modules handle only HTTP concerns (parameter parsing, status codes, response formatting) and delegate all business logic to service modules. This means that if the API framework were changed from FastAPI to another framework, only the route layer would need modification; the service layer would remain untouched.

The Don't Repeat Yourself (DRY) principle was applied through the shared validators module. Rather than duplicating filename validation logic in every service function, a single `validate_filename()` function is called consistently across all endpoints. Similarly, the `require_file_exists()` helper combines filename validation with a filesystem existence check, reducing the chance of inconsistencies between endpoints.

The Single Responsibility Principle was applied at the service level. Each service module handles one domain: `upload_service` manages file storage, `diagnostics_service` manages anomaly detection, `alert_service` manages warning retrieval and acknowledgement, `granite_service` manages AI explanation generation, and `data_service` manages data retrieval. This makes the codebase easier to test, debug, and extend.

Configuration was externalised into `config.py` rather than being hardcoded throughout the codebase. Anomaly thresholds, sensor lists, model settings, and pagination defaults are all defined in one place. This means that if the acceptable range for engine coolant temperature needs to be adjusted, only one value in one file needs to change, rather than hunting through multiple service files.

Section 3: Use Instructions

3.1 Installation and System Requirements

To ensure the system operates efficiently, particularly the local Large Language Model (LLM), the host machine must meet the following hardware and software requirements:

- **Operating System:** macOS (Apple Silicon M1/M2/M3 or Intel), Windows 10/11 or Linux -

Minimum Hardware: 8 GB RAM, multi-core CPU (Intel i5/Ryzen 5 or equivalent) and at least 5 GB of available storage (we recommend 16 GB RAM and an SSD for optimal local AI inference speeds)

- **Prerequisite Software:** Python 3.12+, Node.js (v18+), npm and Ollama

3.2 Deployment and Configuration

The deployment process involves setting up three distinct components: the backend API, the local database and the frontend client and were going to give you the steps:

Step 1: Backend Setup & Database Initialization (correct me please)

Navigate to the Backend directory in your terminal: `cd Backend`

Create and activate a Python virtual environment to isolate dependencies:

- Mac/Linux: `"python3 -m venv.venv"` then `"source.venv/bin/activate"`
- Windows: `"python -m venv.venv"` then `".venv\Scripts\activate"`

Install the required packages: `"pip install fastapi uvicorn pandas numpy pydantic httpx sqlalchemy ollama"`

Step 2: AI Model Deployment

To deploy the local AI securely, launch the Ollama application on your system. Once the application is running in the background, open a terminal and execute the following command to download the specific Granite model: `"ollama run granite3-dense:2b"`

Step 3: Frontend Setup

Go to the root directory and install the Node modules: `"npm install"`

3.3 Launching the Application

Once you're done with the previous part, you can launch the application and perform the initial setup which consist in:

1. Start the backend server by going into the backend directory through the `"cd backend"` command. and run : `"python -m uvicorn main:app --reload"`
2. Open a new terminal within the frontend directory to start the frontend server thrw: `"npm run dev"`
3. access the system by opening a web browser and navigate to: `"http://localhost:5173"` 4. Upon launching, the user will get an authentication screen. Click "Register" to create a new user account. The system will securely hash your password and generate a JWT for your session. Once logged in, you will be redirected to the main diagnostic dashboard where you can

upload your first OBD-II file (reminder: It has to be a csv file)